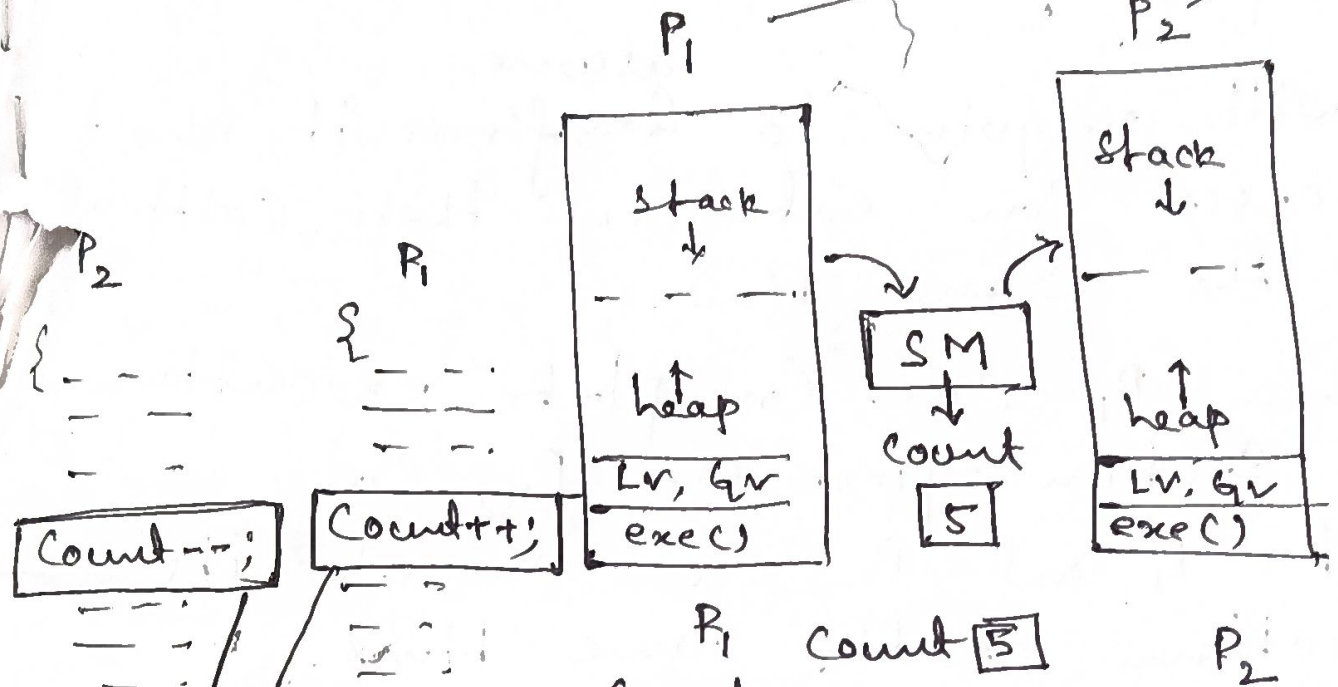
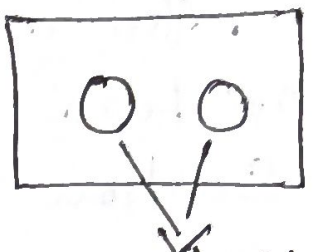
Inter-process Communication

(or)

Process Synchronization

Cooperating process.





Two different process of same system wants to Communicate by using some shared resource of variable

⇓

IPC

P1 Count 5

P2

Count++	Count --
① MOV R0, Count	① MOV R1, Count
② INR R0	② DCR R1
③ MOV Count R0	③ MOV Count, R1

Lets see the different sequence in which Codes of the above two process execute.

- P1: 1, 2, 3 | P2: 1, 2, 3 ⇒ Count 5
- P1: 1, 2 | P2: 1, 2, 3 | P1: 3 ⇒ Count 6 (preempted →)
- P1: 1, 2 | P2: 1, 2 | P1: 3 | P2: 3 ⇒ Count 4 (preempt →, preempt →)

→ Incon-sistent sy

As in RR

Portion of the program where it ~~will~~ accesses any of shared resources of the system is called Critical Section of that program.

The Condition in which output depends on the sequence in which instructions of the program will get executed is called race around Condition

→ So in order to resolve this issue (or) to avoid race around condition we will require to synchronize the execution of two (or) more cooperative processes.

Also we will require to assure it that no two processes can enter in their critical section at the same time.

Means either P_1 can complete execution of its critical section first (or) P_2 can complete it. P_1 and P_2 can execute its critical section at the same time.

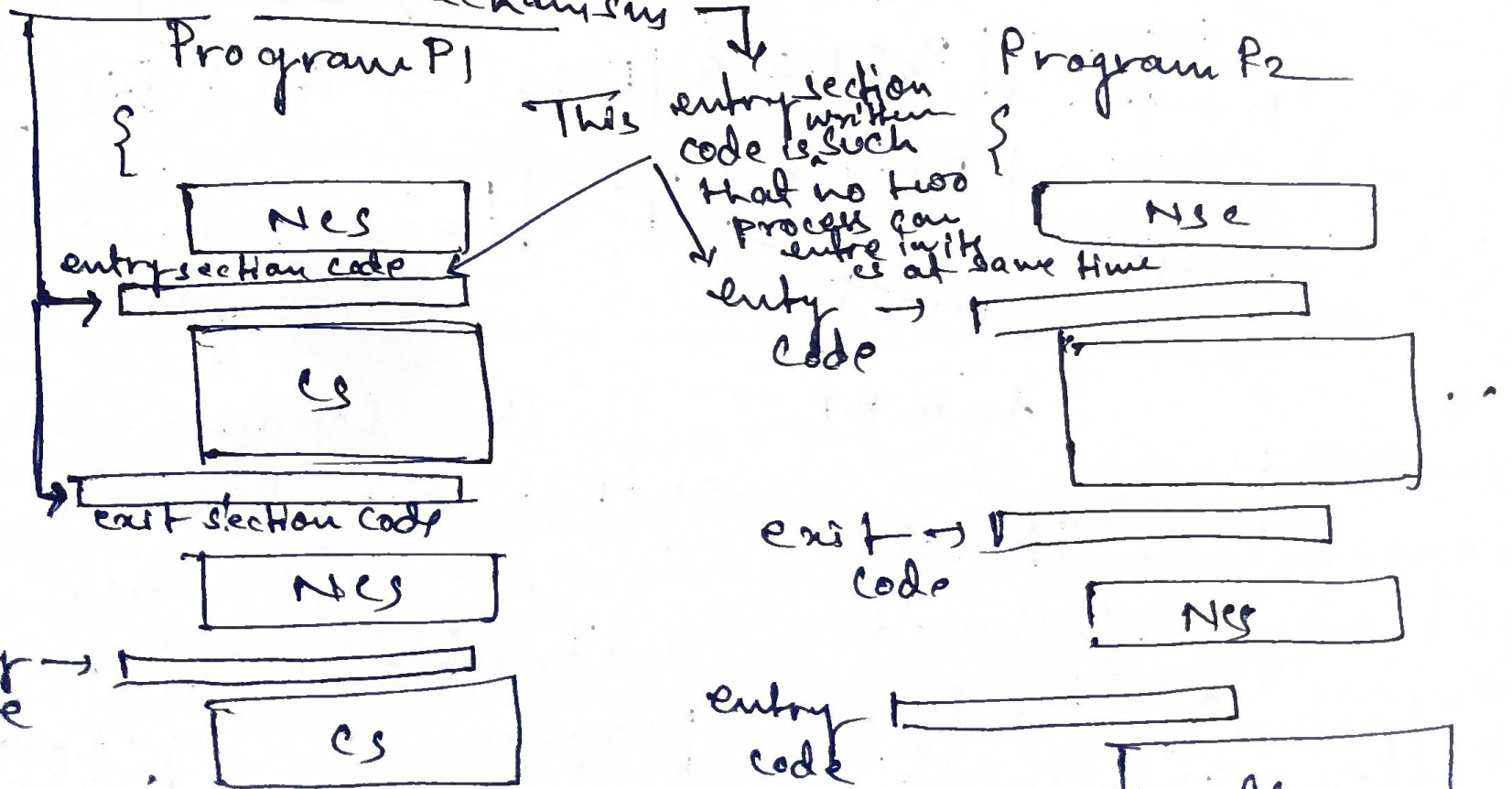
⇩

To assure the above things synchronization is required.

⇩

Means that we will require to specify some sequence of execution of the cooperative processes.

Synchronization mechanism →



By using Synchronization mechanism we will make it possible to execute processes in a certain sequence, so that no two processes can enter in its critical section at the same time.

Requirements of Synchronization mechanism

Primary: 1. Mutual exclusion
2. Progress

Secondary: 3. Bounded Waiting
4. Portability (or) Architectural neutrality

Mutual exclusion → If one process in CS it will not allow other process to enter in its critical section.

Progress → If a process don't want to enter in its critical section, it will not stop other process to enter in its critical section.

→ Condition that every Schronization mechanism should meet. (Mendatory Condition)

Solution to Critical Section problem

Solution to critical section problem must satisfy the following three or four requirements:

1. Mutual Exclusion
2. Progress
3. Bounded Wait

} Mandatory

and

4. Architectural Neutrality

Solution is divided into 2 types according to number of processes

1. Two process solution ⇒ Peterson Algorithm
2. Multi process solution ⇒ Semaphore

Two process solution

First Attempt

```
int turn;
Initially turn = 0;
turn = i;
```

} Shared variable

```
(Process i)
do {
    while (turn != i);
    critical section
    turn = j;
    remainder section
} while (1);
```

```
(Process j)
do {
    while (turn != j);
    critical section
    turn = i;
    remainder section
} while (1)
```

→ ~~This method~~ Both process control execution of other.

→ Therefore, it satisfy mutual exclusion, but not progress.

Second Attempt

boolean flag[2]; → flag[Po Pi]
T F

initially flag[0] = flag[1] = false.

flag[i] = true ⇒ Pi ready to enter in its critical section.

Process Pi

```
do {
    flag[i] = true;
    while (flag[j]);
        critical section
    flag[i] = false
        remainder section
} while(1);
```

Process Pj

```
do {
    flag[j] = true;
    while (flag[i]);
        Critical section
    flag[j] = false;
        remainder section
} while(1);
```

→ Again it satisfy mutual exclusion but not progress.

Now, Peterson algorithm (or) solution Combined shared variable of of first and second attempt.

→ It meets all three requirements and solves the critical section problem for two processes.

Peterson solution

- A classic software-based solution to the critical section problem.
- May not work correctly on modern computer architecture.
- Peterson's solution is restricted to two processes. Let's call the processes P_i and P_j.
- Peterson solution requires two data item to be shared between the two processes: Pi and Pj

int turn ⟶ Indicates whose turn it is to enter its critical section.

boolean flag[2] → used to indicate if a process is (or) ready to enter in its critical section.

Process Pi

```
do {
    flag[i] = true;          → I want to enter
    turn = j;                → Your turn
    while(flag[j] && turn == j);   → If you want to enter and its your turn, I will wait.

    critical section

    flag[i] = false;         → I don't want to enter anymore.

    remainder section;

} while(1);
```

Process j

```
do {
    flag[j] = true;
    turn = i;
    while(flag[i] && turn == i);

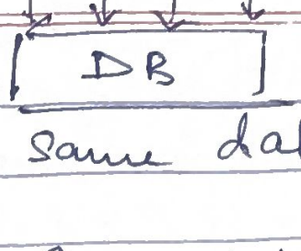
    critical section

    flag[j] = false;

    remainder section;

} while(1);
```

Solution to a Reader Writer Problem



Same database (or) data

R — W → Problem
W — R → Problem
W — W → Problem
R — R → No Problem

int readcount=0 → read count
int rc=0;
Semaphore mutex=1;
readcount=1;

Reader → void Reader (void)

do
{
while (1)
{

while (1)
{

For writer

wait (wrt);

[write operation (critical section)]

signal (wrt);

For Reader

wait (mutex);
readcount++;
if (readcount == 1)
wait (wrt);
signal (mutex);

[Read operation (critical section)]

wait (mutex);
readcount--;
if (readcount == 0)
signal (wrt);
signal (mutex);

7351479874

Reader

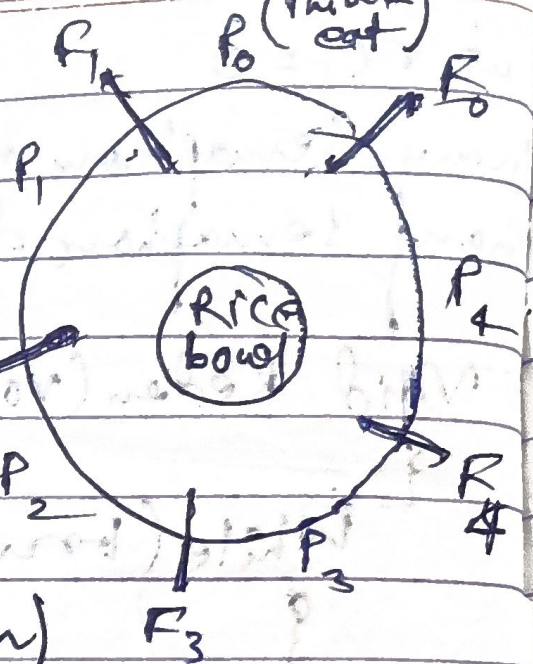
```
int rc = 0
binary semaphore mutex = 1
binary semaphore db = 1

Void Reder (void)
{
  while (true)
  {
    down (mutex)
    rc = rc + 1
    if (rc == 1) then down(db)
    up (mutex)
```

```
down (mutex)
  rc = rc - 1;
if (rc == 0) then up(db)
  up (mutex)
  process Data
  }
}

void writer (void)
{
  while (true)
  {
    down (db)
    CS
    up (db)
  }
}
```

Dining philosphen problem solution




```
Void philosphen (void)
{
  while (true)
  {
    Thinking()
    take fork(i)
    take fork((i+1) modn)
    EAT()
    put fork(i)
    put fork((i+1) modn)
  }
}
```

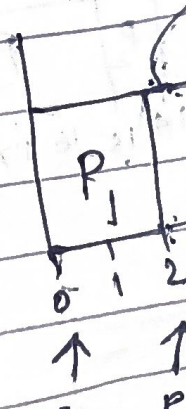
Case

P_0 → f_0 f_1 ; P_1 → f_1 f_2

Process

P_1
P_2
P_3
P_4





Solution → take S[i] fix Semaphore

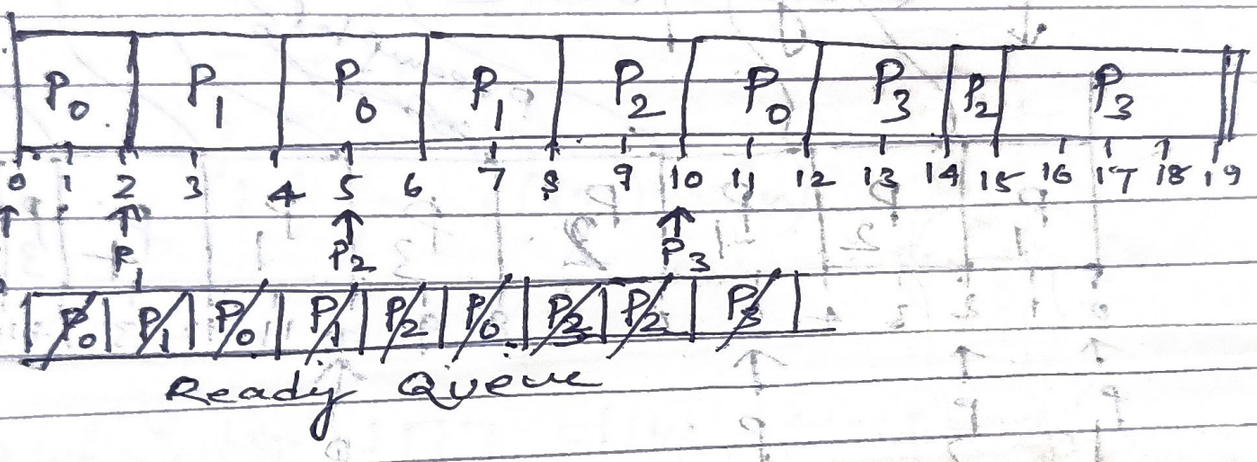
	S_0	S_1	S_2	S_3	S_4
P_0	S_0	S_1			
P_1		S_1	S_2		
P_2			S_2	S_3	
P_3				S_3	S_4
P_4	S_4				S_0

← take this Semaphore

```
void philosphen (void)
{
  while (true)
  {
    Thinking()
    wait to take fork(Si)
    wait to take fork(S(i+1) modn)
    EAT()
    signal put fork(Si)
    signal put fork(S(i+1) modn)
  }
}
```

Process	Burst Time	Arrival Time
P_0	7	0
P_1	4	2
P_2	3	5
P_3	6	10

Gantt Chart. TQ = 2



Process	B.T	A.T
P_0	7	0
P_1	4	2
P_2	3	5
P_3	6	10

Advantages of Semaphores

Some of the advantages of semaphores are as follows:

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores

Some of the disadvantages of semaphores are as follows –

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for last scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low prior ty processes may access the critical section first and high priority processes later.

Producer Consumer Problem using Semaphores (Bounded Buffer):

Problem Statement – We have a buffer of fixed size. A producer can produce an item and can place in the buffer. A consumer can pick items and can consume them. We need to ensure that when a producer is placing an item in the buffer, then at the same time consumer should not consume any item. In this problem, buffer is the critical section.

To solve this problem, we need two counting semaphores – Full and Empty. "Full" keeps track of number of items in the buffer at any given time and "Empty" keeps track of number of unoccupied slots.

Initialization of semaphores –

mutex = 1

Full = 0 // Initially, all slots are empty. Thus full slots are 0

Empty = n // All slots are empty initially

Solution for Producer –

```
do{

//produce an item

wait(empty);

wait(mutex);


//place in buffer


signal(mutex);

signal(full);

}while(true)
```

When producer produces an item then the value of "empty" is reduced by 1 because one slot will be filled now. The value of mutex is also reduced to prevent consumer to access the buffer. Now, the producer has placed the item and thus the value of "full" is increased by 1. The value of mutex is also increased by 1 beacuse the task of producer has been completed and consumer can access the buffer.

Solution for Consumer –

```
do{

wait(full);

wait(mutex);

// remove item from buffer

signal(mutex);

signal(empty);
```

```
// consumes item

}while(true)
```

As the consumer is removing an item from buffer, therefore the value of "full" is reduced by 1 and the value is mutex is also reduced so that the producer cannot access the buffer at this moment. Now, the consumer has consumed the item, thus increasing the value of "empty" by 1. The value of mutex is also increased so that producer can access the buffer now.